

Exercise 1: Implementing the Singleton Pattern

```
src > J Logger.java > ...
1  public class Logger {
2
3      private static Logger instance;
4
5      private Logger() {
6          System.out.println(x: "Logger object created");
7      }
8
9      public static Logger getInstance() {
10         if (instance == null) {
11             instance = new Logger();
12         }
13         return instance;
14     }
15
16     public void log(String message) {
17         System.out.println("LOG: " + message);
18     }
19 }
```

```
src > J SingletonTest.java > ...
1  public class SingletonTest {
2
3      Run | Debug
4      public static void main(String[] args) {
5
6          Logger logger1 = Logger.getInstance();
7          Logger logger2 = Logger.getInstance();
8
9          logger1.log(message: "Application Started");
10         logger2.log(message: "User Logged In");
11
12         if (logger1 == logger2) {
13             System.out.println(x: "Only one Logger instance exists.");
14         } else {
15             System.out.println(x: "Multiple Logger instances exist.");
16         }
17
18         System.out.println("Logger1 HashCode: " + logger1.hashCode());
19         System.out.println("Logger2 HashCode: " + logger2.hashCode());
20     }
21 }
```

```
PS D:\LOCAL\New folder\SingletonPatternExample> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\LOCAL\New folder\SingletonPatternExample\bin' 'SingletonTest'
Logger object created
LOG: Application Started
LOG: User Logged In
Only one Logger instance exists.
Logger1 HashCode: 1450495309
Logger2 HashCode: 1450495309
```

Exercise 2: Implementing the Factory Method Pattern

```
src > J Document.java > Document
1 public interface Document {
2     void open();
3 }
```

```
src > J ExcelDocument.java > ExcelDocument
1 public class ExcelDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening Excel Document");
6     }
7 }
```

```
src > J PdfDocument.java > PdfDocument
1 public class PdfDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening PDF Document");
6     }
7 }
```

```
src > J WordDocument.java > WordDocument
1 public class WordDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening Word Document");
6     }
7 }
```

```
src > J DocumentFactory.java > DocumentFactory
1 public abstract class DocumentFactory {
2
3     public abstract Document createDocument();
4
5     public void openDocument() {
6         Document doc = createDocument();
7         doc.open();
8     }
9 }
```

```
src > J ExcelDocumentFactory.java > ExcelDocumentFactory
1 public class ExcelDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new ExcelDocument();
6     }
7 }
```

src > PdfDocumentFactory.java > PdfDocumentFactory

```
1 public class PdfDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new PdfDocument();
6     }
7 }
```

src > WordDocumentFactory.java > WordDocumentFactory

```
1 public class WordDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new WordDocument();
6     }
7 }
```

src > FactoryMethodTest.java > FactoryMethodTest

```
1 public class FactoryMethodTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         DocumentFactory wordFactory = new WordDocumentFactory();
6         wordFactory.openDocument();
7
8         DocumentFactory pdfFactory = new PdfDocumentFactory();
9         pdfFactory.openDocument();
10
11        DocumentFactory excelFactory = new ExcelDocumentFactory();
12        excelFactory.openDocument();
13    }
14 }
```

```
Opening Word Document
Opening PDF Document
Opening Excel Document
```